Análise Comparativa de Estratégias de Controle Populacional em Algoritmos Genéticos aplicados ao Problema das N-Rainhas

Vinícius Rubin[1], **Igor Yepes**[1]

[1]Instituto de Educação, Ciência e Tecnologia Farroupilha - Campus Frederico Westphalen (IFFAR-FW)

Abstract. *This article investigates how different population control strategies affect the performance of genetic algorithms when solving the n-queens problem. We describe the design of a baseline implementation, outline the experimental protocol, and discuss how metrics such as execution time, number of generations, and success rate will be interpreted. The study aims to identify configurations that preserve diversity while accelerating convergence, contributing guidelines for practitioners who deploy evolutionary heuristics in combinatorial search tasks.*

Resumo. *Este artigo investiga como diferentes estratégias de controle populacional influenciam o desempenho de algoritmos genéticos na resolução do problema das n-rainhas. São descritos o planejamento da implementação base, o protocolo experimental e a forma de análise de métricas como tempo de execução, número de gerações e taxa de sucesso. O estudo busca apontar configurações que preservem a diversidade e acelerem a convergência, oferecendo subsídios para o emprego de heurísticas evolutivas em tarefas de busca combinatória.*

1. Introdução

Algoritmos Genéticos (AGs) são heurísticas evolutivas baseadas em seleção natural, amplamente utilizadas para otimização em espaços de busca complexos [Eiben and Smith 2015]. A capacidade de combinar exploração estocástica com operadores simples de reprodução torna esses algoritmos versáteis em aplicações que vão de problemas combinatórios a tarefas de engenharia e inteligência artificial.

Entre os elementos que influenciam o desempenho dos AGs, o *controle populacional* — isto é, o modo como indivíduos são mantidos, removidos ou substituídos ao longo das gerações — desempenha papel central. Estratégias como substituição total, substituição parcial, elitismo e abordagens baseadas em ciclos de vida têm sido utilizadas na prática, mas a literatura apresenta resultados heterogêneos, conduzidos em contextos experimentais distintos e com ausência de padronização. Como consequência, ainda é difícil comparar, de forma direta e controlada, o impacto real de cada mecanismo no comportamento do algoritmo.

Essa lacuna motiva o presente estudo, cujo objetivo é **comparar sistematicamente diferentes estratégias de controle populacional em um mesmo cenário experimental**, mantendo constantes todos os demais componentes do AG (representação, seleção, cruzamento e mutação). Para isso, utiliza-se o problema das n-rainhas como

ambiente de teste, um benchmark clássico pela sua estrutura simples, escalabilidade controlada e ampla adoção em estudos de otimização.

A contribuição deste trabalho é dupla: (i) oferecer uma avaliação empírica rigorosa de quatro mecanismos de controle populacional — substituição total, substituição parcial, elitismo e uma abordagem baseada em idade — sob condições experimentais uniformes; e (ii) analisar como cada estratégia afeta taxa de sucesso, esforço evolutivo e custo computacional à medida que a dificuldade do problema aumenta.

Os experimentos são conduzidos com múltiplas execuções independentes para cada configuração, permitindo estimar médias, variâncias e intervalos de confiança, além de aplicar testes estatísticos para comparação entre estratégias. Essa estrutura experimental fornece evidências quantitativas que auxiliam na escolha de mecanismos mais adequados para AGs aplicados a problemas combinatórios semelhantes.

2. Fundamentação Teórica

2.1. Algoritmos Genéticos

Algoritmos Genéticos (AGs) são meta-heurísticas inspiradas em princípios de evolução biológica, nas quais uma população de soluções candidatas é iterativamente modificada por operadores de seleção, cruzamento e mutação [Eiben and Smith 2015, Holland 1975]. Cada indivíduo é avaliado por uma função de aptidão (*fitness*), e o processo evolutivo tende a favorecer configurações de melhor desempenho.

O comportamento de um AG é fortemente influenciado por três grupos de decisões: (i) representação dos indivíduos, (ii) função de avaliação e (iii) desenho dos operadores e da política de substituição [de Lacerda and de Leon Ferreira de Carvalho 1999]. Enquanto representação e operadores de cruzamento/mutação são amplamente discutidos na literatura, o *controle populacional* — isto é, a forma como a população é renovada e mantida ao longo das gerações — recebe tratamento menos sistemático, embora esteja diretamente ligado à diversidade, à convergência e ao risco de aprisionamento em ótimos locais.

Neste trabalho, representação, função de aptidão, seleção, cruzamento e mutação são mantidos fixos, e o foco recai justamente sobre o componente de controle populacional, comparando diferentes mecanismos de substituição em um cenário experimental comum.

2.2. O Problema das *n*-Rainhas

O problema das *n*-rainhas consiste em posicionar *n* rainhas em um tabuleiro de xadrez de dimensões *n*×*n* de forma que nenhuma delas ataque outra, o que implica ausência de conflitos em linhas, colunas e diagonais [Bell and Stevens 2009]. Apesar da descrição simples, o número de configurações possíveis cresce rapidamente com *n*, tornando abordagens exaustivas impraticáveis para valores mais elevados.

Estudos como o de Ali et al. [Ali et al. 2023] mostram que técnicas clássicas de busca (BFS, DFS) são eficazes apenas para instâncias pequenas, perdendo viabilidade à medida que o tabuleiro cresce. Em contrapartida, o problema possui estrutura bem definida, múltiplas soluções conhecidas e um parâmetro de dificuldade (n) facilmente ajustável. Por isso, é amplamente adotado como *benchmark* em experimentos com

heurísticas e meta-heurísticas, incluindo AGs [Sarkar and Nag 2017, Farhan et al. 2015, Ali et al. 2023].

Neste estudo, o problema das *n*-rainhas é utilizado como ambiente de teste justamente por combinar simplicidade conceitual, escalabilidade controlada e ampla presença na literatura, o que facilita a comparação dos resultados obtidos com trabalhos anteriores.

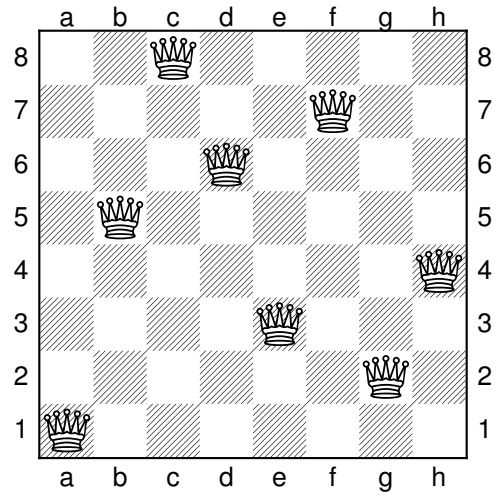


Figura 1. Exemplo de solução válida para o problema das 8-rainhas.

2.3. Uso da Linguagem Python e Bibliotecas Relacionadas

A implementação dos experimentos é realizada em `Python`, linguagem amplamente utilizada em computação científica devido à clareza sintática e à disponibilidade de bibliotecas numéricas e de visualização [Python Software Foundation 2025a]. A biblioteca `NumPy` é empregada para representar populações e operar sobre vetores e matrizes de forma vetorizada [Harris et al. 2020], enquanto o gerador `numpy.random` é utilizado nas operações estocásticas de inicialização, seleção e aplicação de operadores [Python Software Foundation 2025b, Harris et al. 2020]. A medição de tempo é feita com o módulo `time` [Python Software Foundation 2025c], e a síntese gráfica dos resultados é conduzida com `matplotlib` [Hunter 2007].

Esse conjunto de ferramentas oferece suporte suficiente para implementar o AG base, alternar entre diferentes estratégias de controle populacional e registrar métricas de desempenho de forma reprodutível, evitando que questões de infraestrutura ofusquem o foco principal do estudo.

2.4. Trabalhos Relacionados

Diversos trabalhos utilizam o problema das *n*-rainhas como *benchmark* para avaliar variações em AGs, explorando principalmente representação e controle de diversidade. Nesta subseção, destacam-se estudos que motivam o desenho experimental adotado aqui e ajudam a situar a contribuição deste trabalho, que se volta especificamente ao impacto de diferentes estratégias de controle populacional.

2.4.1. Representações Binária e Inteira para o Problema das *n*-Rainhas [Richter and Berlezi 2012]

Richter et al. [Richter and Berlezi 2012] comparam duas representações genéticas para o problema das *n*-rainhas: uma codificação binária e uma codificação inteira do tipo *InOrder*. A primeira exige conversão de bits para posições de rainhas e é suscetível a colisões em linhas; a segunda mapeia diretamente cada coluna a uma linha distinta, evitando conflitos de linha e coluna por construção.

Os resultados mostram que a representação *InOrder* reduz o número de soluções inviáveis e acelera a convergência, sobretudo para tabuleiros maiores. Embora os autores mantenham o controle populacional em um esquema tradicional, seu trabalho reforça a importância de fixar uma representação adequada antes de comparar outras variações de um AG. Neste estudo, adotamos uma representação por permutação semelhante à *InOrder* justamente para minimizar o impacto de soluções inviáveis e isolar o efeito das estratégias de substituição populacional.

2.4.2. Comparação entre Algoritmo Genético e Buscas Clássicas (BFS/DFS) [Ali et al. 2023]

Ali et al. [Ali et al. 2023] comparam um AG com buscas clássicas (BFS/DFS) no problema das *n*-rainhas, analisando a escalabilidade de cada método. Utilizando uma representação vetorial em que cada posição do cromossomo indica a linha ocupada por uma rainha, os autores observam que, para instâncias pequenas, BFS e DFS são mais eficientes em tempo de execução, mas o AG passa a ser mais competitivo à medida que o tamanho do tabuleiro aumenta.

Embora o foco do estudo seja a comparação entre paradigmas (busca exaustiva vs. AG), os resultados reforçam o papel dos AGs como alternativa viável para instâncias maiores e justificam seu uso como meta-heurística de referência neste trabalho. Diferentemente de Ali et al., aqui não comparamos AGs com métodos clássicos de busca, mas exploramos, dentro do próprio paradigma evolutivo, como diferentes políticas de controle populacional afetam desempenho em um problema combinatório já consolidado na literatura.

2.4.3. Algoritmo Genético Adaptativo com Controle de Diversidade [Sarkar and Nag 2017]

Sarkar e Nag [Sarkar and Nag 2017] propõem um AG adaptativo para o problema das *n*-rainhas, enfatizando mecanismos explícitos de controle da diversidade populacional. Os autores utilizam uma representação por permutação, um cruzamento do tipo *Order 1 Crossover* e operadores de mutação com probabilidades relativamente altas, além de estruturas auxiliares (como um repositório de indivíduos menos aptos) para evitar estagnação em ótimos locais.

Os resultados indicam que o esquema adaptativo encontra soluções ótimas para valores elevados de N (até 100), sugerindo que a forma como a população é renovada

tem influência direta na qualidade e na velocidade da convergência. O trabalho, porém, combina várias alterações simultâneas (função de aptidão, mutação, repositório, etc.), o que dificulta isolar o efeito de cada componente. Em contraste, o presente estudo mantém constantes representação e operadores genéticos e varia apenas a política de controle populacional, buscando medir de forma mais direta o impacto da substituição e do envelhecimento sobre o desempenho do AG.

2.4.4. Busca de Todas as Soluções Válidas para as 8-Rainhas [Farhan et al. 2015]

Farhan et al. [Farhan et al. 2015] utilizam um AG para encontrar as 92 soluções válidas do problema das 8-rainhas, empregando uma representação vetorial que elimina conflitos em linhas e colunas e concentra a avaliação em ataques diagonais. A função de aptidão é definida em termos do número de conflitos diagonais, e o AG é executado até que todas as soluções distintas sejam encontradas.

Embora o objetivo seja diferente do deste trabalho (cobertura do espaço de soluções em vez de análise comparativa de políticas de controle), o estudo ilustra a capacidade dos AGs de explorar um espaço combinatório rico e de encontrar múltiplas soluções válidas. Aqui, aproveitamos essa mesma característica do problema, mas direcionamos a atenção para como diferentes estratégias de substituição e um esquema de controle por idade alteram a taxa de sucesso e o esforço necessário para encontrar ao menos uma solução válida em instâncias progressivamente mais difíceis.

2.5. Controle Populacional Baseado em Idade

Entre as estratégias possíveis de controle populacional, abordagens que incorporam algum tipo de envelhecimento ou tempo de vida vêm sendo exploradas como forma de equilibrar exploração e intensificação, evitando que indivíduos muito antigos dominem a população indefinidamente. O controle por idade adotado neste trabalho segue essa perspectiva, mas em uma forma simples e acoplada diretamente à aptidão atual dos indivíduos.

Cada indivíduo possui uma idade e um *lifetime* (tempo máximo de vida). A cada geração, a idade é incrementada em uma unidade; quando a idade ultrapassa o lifetime, o indivíduo é removido. O tamanho da população é mantido fixo: após a remoção dos indivíduos cujo tempo de vida se esgotou, novos descendentes são gerados para preencher as vagas.

2.5.1. Cálculo do Lifetime

O tempo de vida máximo é recalculado a cada geração com base na distribuição de aptidões daquela população. Seja $f_{\min}$ a melhor aptidão da geração, $f_{\max}$ a pior e $f(i)$ a aptidão do indivíduo i. O lifetime é dado por:

$$\text{lifetime}(i) = L_{\min} + \left\lfloor (L_{\max} - L_{\min}) \cdot \frac{f_{\max} - f(i)}{f_{\max} - f_{\min}} \right\rfloor,$$

de modo que indivíduos mais aptos tendem a viver por mais gerações que in-

divíduos menos aptos. Os valores utilizados neste trabalho são

$$L_{\min} = 1, \qquad L_{\max} = 5,$$

escolhidos empiricamente para induzir renovação contínua sem tornar a população excessivamente instável.

2.5.2. Reprodução e Manutenção do Tamanho Populacional

Após a remoção dos indivíduos cujo tempo de vida se esgotou, seja S o número de sobreviventes. O número de descendentes a serem criados é definido por

$$N_{\text{novos}} = \lceil \text{taxa}_{\text{reprodução}} \cdot S \rceil ,$$

com

$$\text{taxa}_{\text{reprodução}} = 0{,}5.$$

Os novos indivíduos são gerados a partir dos operadores padrão do AG (seleção em torneio binário, cruzamento OX e mutação por troca), iniciam com idade zero e têm seus lifetimes calculados após a avaliação. Se a combinação de sobreviventes e descendentes ainda não atingir o tamanho populacional nominal, indivíduos adicionais são reinicializados aleatoriamente.

2.5.3. Resumo e Relação com Este Estudo

O mecanismo de idade adotado pode ser resumido em três passos: (i) incrementar a idade de todos os indivíduos; (ii) remover aqueles cuja idade exceda o lifetime; (iii) gerar novos descendentes até restaurar o tamanho populacional. Ao vincular o lifetime à aptidão relativa de cada geração, o algoritmo promove uma renovação constante, preservando bons indivíduos por mais tempo, mas evitando que qualquer cromossomo permaneça indefinidamente na população.

Essa estratégia fornece um contraste direto com os esquemas de substituição total, substituição parcial e elitismo investigados neste trabalho, permitindo comparar, sob as mesmas condições experimentais, um controle populacional baseado em idade com políticas clássicas que não incorporam envelhecimento explícito. A metodologia detalhada na Seção 3 parte dessa fundamentação para definir os cenários de comparação empírica entre as quatro estratégias.

3. Metodologia

Este estudo compara o impacto de quatro estratégias de controle populacional em algoritmos genéticos (AGs) aplicados ao problema das *n*-rainhas, seguindo diretrizes de experimentação reprodutível em computação evolutiva [Eiben and Smith 2015]. Em todas as configurações, representação, função de aptidão e operadores genéticos permanecem fixos, permitindo isolar o efeito da política de substituição.

3.1. Algoritmo Genético Base

Cada indivíduo é representado por uma permutação de tamanho n, eliminando conflitos em linhas e colunas. A aptidão corresponde ao número de pares de rainhas em conflito diagonal, método de avaliação comum na literatura e com complexidade $O(n^2)$.

A população inicial é gerada embaralhando sequências de 0 a $n-1$. A evolução utiliza seleção por torneio binário, cruzamento *Order Crossover* (OX) e mutação por troca de duas posições. A taxa de mutação é mantida fixa em **10%** em todas as estratégias, reduzindo a dimensionalidade do espaço experimental e possibilitando concentrar a análise exclusivamente nos mecanismos de controle populacional.

3.2. Estratégias de Controle Populacional

Quatro mecanismos são comparados:

- **Substituição total**: toda a população é renovada a cada geração.
- **Substituição parcial**: uma fração dos indivíduos é preservada e o restante substituído.
- **Elitismo**: uma porcentagem dos melhores indivíduos é garantida na nova geração.
- **Controle por idade**: indivíduos possuem tempo de vida limitado, calculado a partir da aptidão, sendo removidos quando ultrapassam seu *lifetime*.

Os tamanhos populacionais avaliados (100, 200, 300) representam três escalas típicas na prática de AGs — pequena, média e maior — permitindo observar como a quantidade de indivíduos interage com cada política de substituição.

A Tabela 1 resume os parâmetros específicos de cada mecanismo.

Tabela 1. Configurações avaliadas por estratégia de controle populacional

Estratégia	Parâmetros	Valores
Substituição total	Tamanho da população	100, 200, 300
	Taxa de mutação	10%
Substituição parcial	Fração substituída	25%, 50%, 75%
	Tamanho da população	100, 200, 300
Elitismo	Fração de elite	10%, 20%, 30%
	Tamanho da população	100, 200, 300
Controle por idade	Lifetime	$L_{\min} = 1$, $L_{\max} = 5$
	Taxa de reprodução	0,5
	Tamanho da população	100, 200, 300

3.3. Controle por Idade no Contexto Experimental

O mecanismo utilizado neste trabalho segue o modelo descrito na Seção 2. Após a avaliação da população, o lifetime de cada indivíduo é recalculado com base no intervalo de aptidões; a idade aumenta uma unidade; e indivíduos cuja idade ultrapassa o lifetime são removidos. Em seguida, são criados

$$N_{\text{novos}} = \lceil 0.5 \cdot S \rceil \,,$$

onde S é o número de sobreviventes, complementando-se a população com reinicializações aleatórias quando necessário. Representação, seleção, cruzamento e mutação permanecem idênticos aos das demais estratégias.

3.4. Experimentos

Para cada combinação de:

- estratégia de controle populacional,
- tamanho da população (100, 200, 300),

são realizadas **100 execuções independentes** para cada valor de n. A taxa de mutação é mantida fixa em **10%** em todas as condições experimentais, seguindo recomendações da literatura para representações permutacionais e reduzindo o volume total de cenários, o que permite concentrar a análise exclusivamente no impacto das estratégias de controle populacional.

Critério de parada

Cada execução é encerrada quando:

- uma solução válida (aptidão zero) é encontrada; ou
- o algoritmo atinge o limite de **1.000 gerações**.

Esse limite é amplamente utilizado em estudos experimentais com AGs e atua como barreira computacional uniforme, evitando que execuções divergentes distorçam as métricas de tempo e esforço evolutivo. Não são utilizados critérios baseados em taxa de sucesso ou limiares arbitrários.

Métricas coletadas

Para cada execução, registram-se:

- **Tempo de execução**;
- **Número de gerações até a solução ou até o limite**;
- **Indicador de sucesso** (solução encontrada dentro de 1.000 gerações).

3.5. Ambiente de Execução e Análise Estatística

Os experimentos são implementados em `Python` utilizando `numpy`, `numba`, `matplotlib`, `argparse` e `psutil`. A geração de aleatoriedade utiliza `numpy.random.default_rng`, com registro explícito da semente para garantir reprodutibilidade.

4. Desenvolvimento

4.1. Arquitetura da Ferramenta Experimental

4.2. Implementação dos Operadores Genéticos

4.3. Execução dos Experimentos

4.4. Organização dos Dados e Pré-processamento

5. Resultados e Discussão

6. Conclusão e Trabalhos Futuros

6.1. Conclusões Gerais

6.2. Limitações do Estudo

6.3. Trabalhos Futuros

Referências

Ali, H. et al. (2023). Performance comparison of genetic algorithms with traditional search techniques on the n-queen problem. In *Proceedings of ICIT*.

Bell, J. and Stevens, B. (2009). A survey of known results and research areas for n-queens. *Discrete Mathematics*, 309(1):1–31.

de Lacerda, E. G. M. and de Leon Ferreira de Carvalho, A. C. P. (1999). Introdução aos algoritmos genéticos. In *Sistemas inteligentes: aplicações a recursos hídricos e sistemas ambientais*. Ed. Universidade / UFRGS / ABRH, Porto Alegre.

Eiben, A. E. and Smith, J. E. (2015). *Introduction to Evolutionary Computing*. Natural Computing Series. Springer, Berlin Heidelberg, 2 edition.

Farhan, A. S., Tareq, W. Z., and Awad, F. H. (2015). Solving n queen problem using genetic algorithm. *International Journal of Computer Applications*, 122(12).

Harris, C. R., Millman, K. J., van der Walt, S. J., Gommers, R., Virtanen, P., Cournapeau, D., Wieser, E., Taylor, J., Berg, S., Smith, N. J., et al. (2020). Array programming with NumPy. *Nature*, 585(7825):357–362.

Holland, J. H. (1975). *Adaptation in Natural and Artificial Systems*. University of Michigan Press.

Hunter, J. D. (2007). Matplotlib: A 2d graphics environment. *Computing in Science & Engineering*, 9(3):90–95.

Python Software Foundation (2025a). *The Python Language Reference, version 3.x*. Python Software Foundation. Acesso em: 28 jun. 2025.

Python Software Foundation (2025b). *Python Module Reference: random — generate pseudo-random numbers*. Python Software Foundation, versão 3.13.5 edition. Acesso em: 28 jun. 2025.

Python Software Foundation (2025c). *Python Module Reference: time — Time access and conversions*. Python Software Foundation, versão 3.13.5 edition. Acesso em: 28 jun. 2025.

Richter, H. and Berlezi, R. (2012). Algoritmo genético aplicado ao problema das n rainhas. *Anais da UNIJUÍ*. Relatório técnico disponível em proceedings.sbmac.

Sarkar, U. and Nag, S. (2017). An adaptive genetic algorithm for solving n-queens problem. *arXiv preprint*.